

I used Cursor's built-in copilot to build both the python program and the static webpage. My process for this assignment was simply describing the game, constraints, and what the finished product should look like and do. I also reviewed, tested and asked for changes when needed. An example of a prompt that I used was the first one of the conversation. I stated the rules of the game, described who I was and what my needs were, and was specific about the fact that each card value be used exactly once and that `eval()` was unsafe on raw user input. I also found it helpful to provide the AI assistant with examples given in the assignment pdf, so it knows things like format and what should and should not happen. A problem that I encountered was in the webpage portion of the assignment. The first browser check showed only one card on the felt table, even though four cards existed in the page. The deal animation used ``animation-fill-mode: both`` and a staggered delay, so cards 2–4 stayed at opacity 0 until their delay finished. This made them never appear in the test browser. I had the coding assistant drop that fill mode, keep cards visible by default, and respect reduced motion. An instance in which I tested the assistant is when I had both solvers run on 200 random 52-card deals. The python program found a solution on 159 hands and reported 41 unsolvable. However, the JavaScript program agreed on every hand, and every printed solution was fed back through the judge and accepted. Known dead hands such as ``1 1 1 1`` and ``2 2 2 2`` correctly have no solution. Overall, vibe coding is very productive as long as you are direct and pay attention to what is being produced.